STM32基础 - GPIO输入与输出

课程目标

1. 熟悉GPIO基本原理
2. 熟悉GPIO输出控制
3. 熟悉GPIO输入控制
4. 熟悉面包板的基本使用
5. 掌握LED灯、蜂鸣器、按键、光敏传感器的基本元器件的控制

课程试验

1. 利用STM32完成基本的LED灯闪烁试验
2. 利用STM32完成按键控制LED灯试验
3. 利用STM32完成光敏传感器控制蜂鸣器试验

课堂引入

我们已经完成了开发工具的部署和工程项目的搭建，那么我们用STM32做点什么呢？咱们一切就从点个灯开始吧！

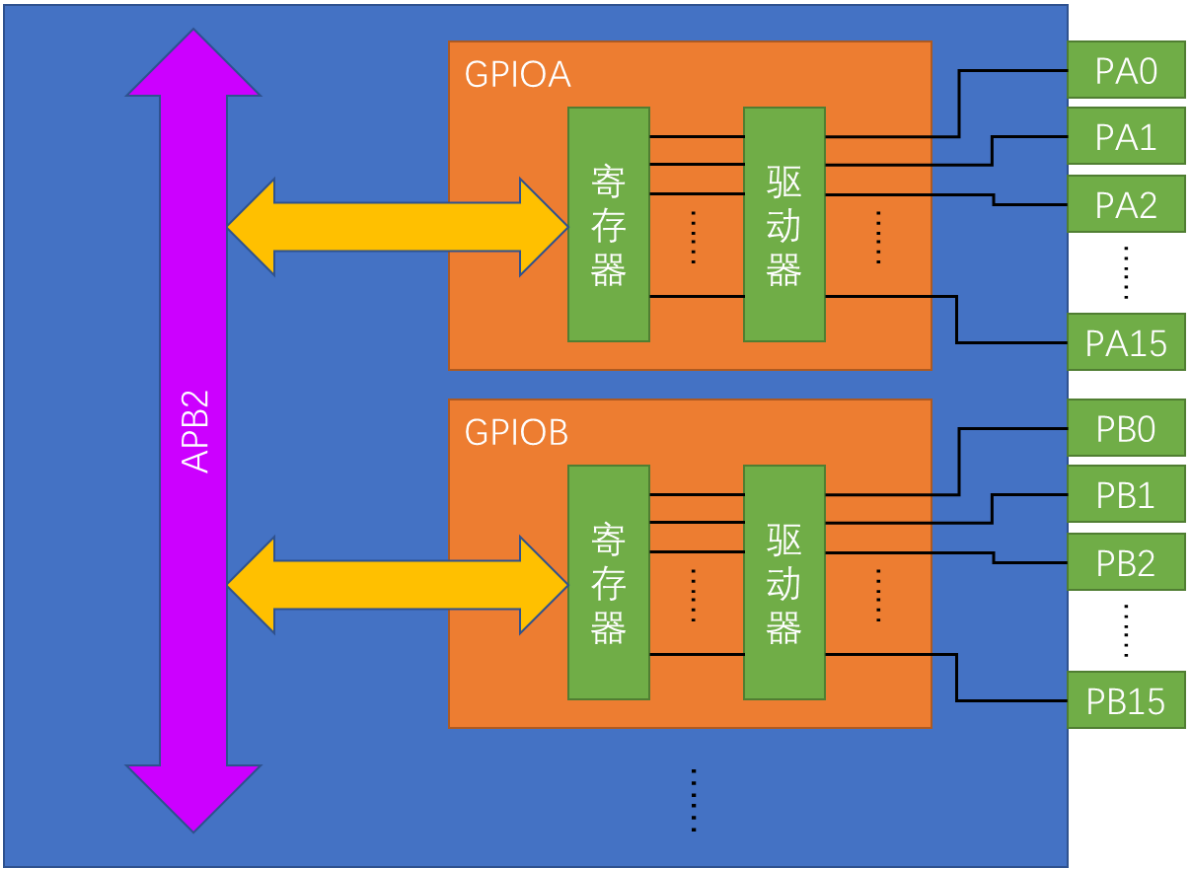
授课进程

一、GPIO

1、简介

GPIO（General Purpose Input Output）通用输入输出口，也就是我们俗称的基本IO口。GPIO是STM32最常用的设备之一。它每个引脚都可以有0~3.3V的电压输出。数据0就是低电平，也就是0伏；数据1就是高电平，也就是3.3伏，最大可以承受5V的电压。GPIO可以配置8种输入输出工作模式，其中输出模式下可控制端口输出高低电平，用以驱动LED、控制蜂鸣器、模拟通信协议输出时序（比如I2C、SPI或者某个芯片特定的协议），而输入模式下可读取端口的高低电平或电压，用于读取按键输入、外接模块电平信号输入、ADC电压采集、模拟通信协议接收数据等。这个输入模式最常见的就是读取按键，用来捕获比如按键按下的事件，此外也可以读取带有数字输出的一些模块，比如我们套件里的光敏电阻模块、热敏电阻模块等。

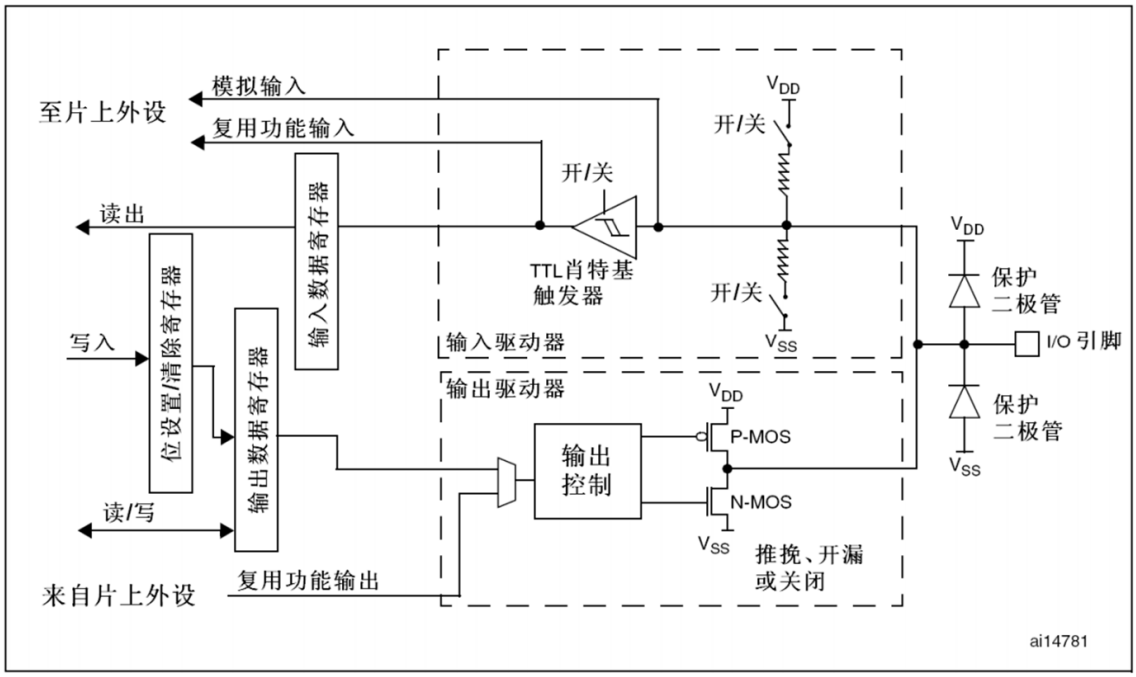
2、基本结构



这个框图这就是GPIO的整体构造，其中左边的是APB2外设总线，在STM32中所有的GPIO都是挂载在APB2外设总线上的。其中GPL外设的名称是按照GPIOA、GPIOB、GPIOC等等这样来命名的。每个GPIO外设总共有16个引脚，编号是从0-15。GPIOA的第0号引脚我们一般把它称作PA0，接着第1号就是PA1，然后PA2，依次类推一直到PA15。GPIOB也是一样，从PB01一直到PB15这样来命名的。在每个GPIO模块内主要包含了寄存器和驱动器等，寄存器就是一段特殊的存储器，内核可以通过APB2总线对寄存器进行读写，这样就可以完成输出电平和读取电平的功能了。这个寄存器的每一位对应一个引脚，当向输出寄存器写1时，对应的引脚就会输出高电平；写0时，对应的引脚就会输出低电平。输入寄存器读取为1就证明对应的引脚端口目前是高电平，读取为0就意味着此时引脚端口为低电平。STM32内部的GPIO寄存器都是32位的，但是GPIO端口只有16个，所以这个寄存器只有低16位对应的有端口，高16位是没有用到的。图中的驱动器是用来增加信号的驱动能力的，寄存器只负责存储数据。如果要进行点亮LED灯这样的操作的话，还是需要驱动器来负责增大驱动能力的。这就是GPIO的基本结构。

3、GPIO端口位结构

图13　I/O端口位的基本结构



这是STM32手册中GPIO中每一位的具体电路结构图。

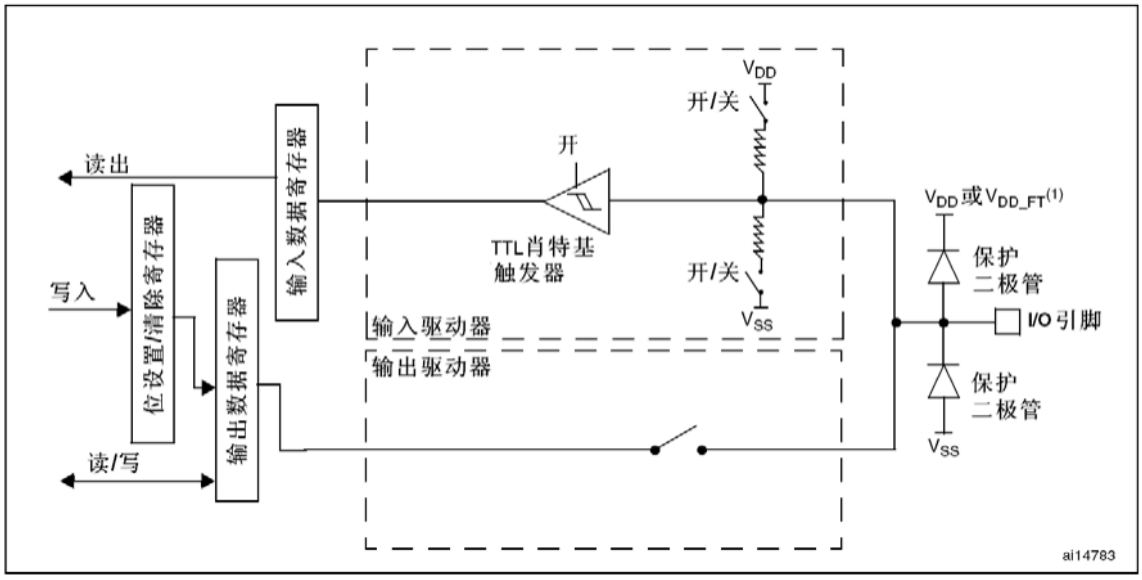
4、GPIO八种输入输出模式

通过配置GPIO的端口配置寄存器，端口可以配置成以下8种模式。

模式名称	性质	模式选择	特征
浮空输入	数字输入	GPIO_Mode_IN_FLOATING（Input floating）	可读取引脚电平，若引脚悬空，则电平不确定
上拉输入	数字输入	GPIO_Mode_IPU（Input pull-up）	可读取引脚电平，内部连接上拉电阻，悬空时默认高电平
下拉输入	数字输入	GPIO_Mode_IPD（Input-pull-down）	可读取引脚电平，内部连接下拉电阻，悬空时默认低电平
模拟输入	模拟输入	GPIO_Mode_AIN（Analog input）	GPIO无效，引脚直接接入内部ADC

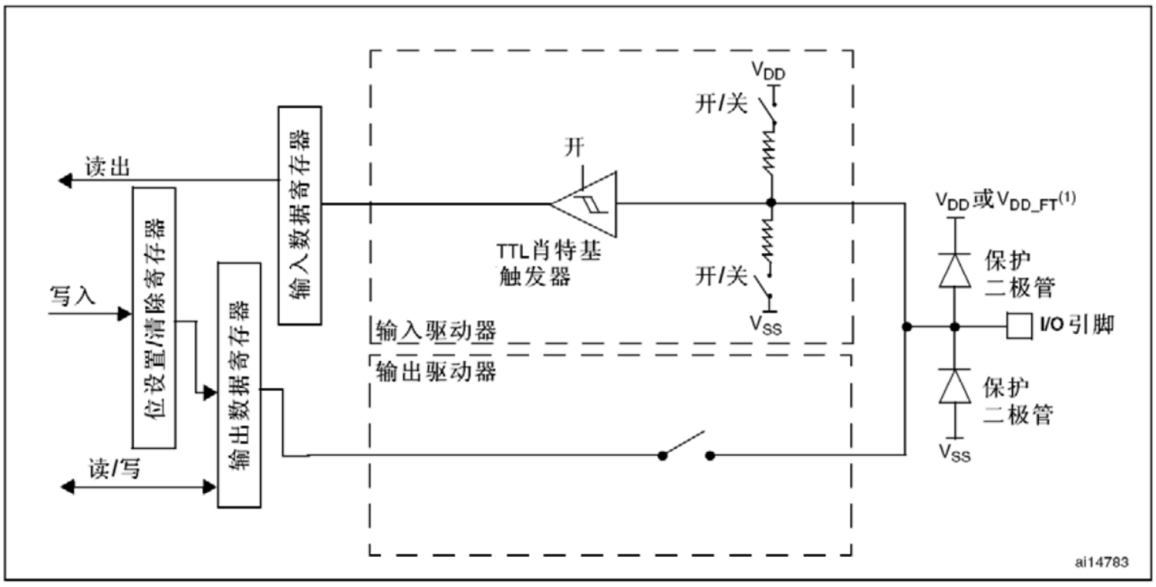
模式名称	性质	模式选择	特征
开漏输出	数字输出	GPIO_Mode_Out_OD（output open-Drain）	可输出引脚电平，高电平为高阻态，低电平接VSS
推挽输出	数字输出	GPIO_Mode_Out_PP（output Push-Pull）	可输出引脚电平，高电平接VDD，低电平接VSS
复用开漏输出	数字输出	GPIO_Mode_AF_OD（Alternate function open-drain）	由片上外设控制，高电平为高阻态，低电平接VSS
复用推挽输出	数字输出	GPIO_Mode_AF_PP（Alternate function push-pull）	由片上外设控制，高电平接VDD，低电平接VSS

图15　输入浮空/上拉/下拉配置



(1) V_{DD_FT} 对5伏容忍I/O脚是特殊的，它与V_{DD}不同

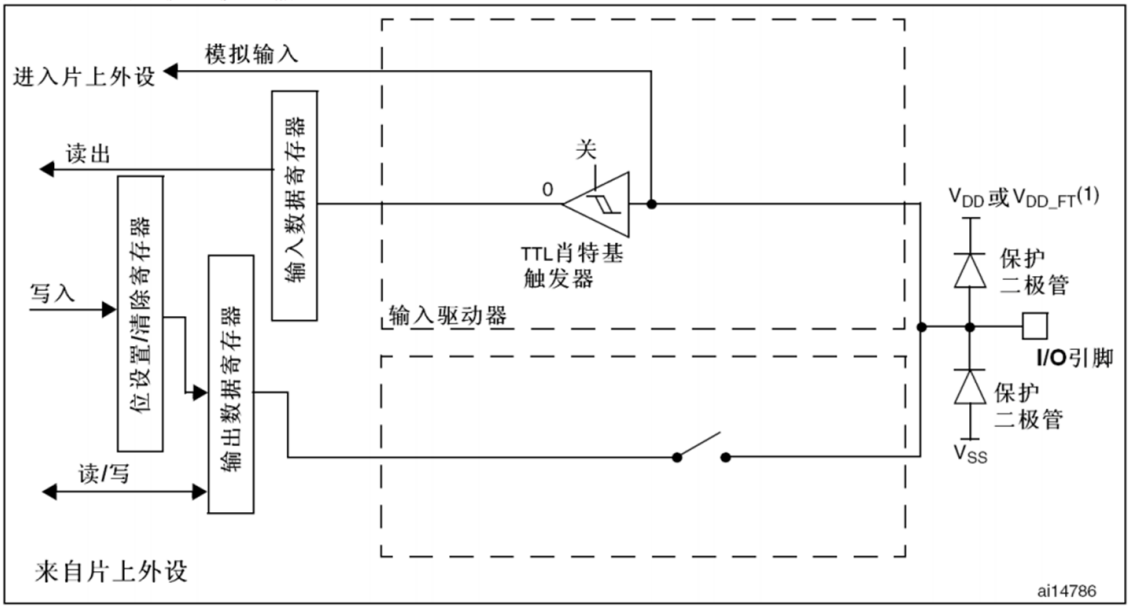
图15　输入浮空/上拉/下拉配置



(1) V_{DD_FT} 对5伏容忍I/O脚是特殊的，它与V_{DD}不同

6、模拟输入

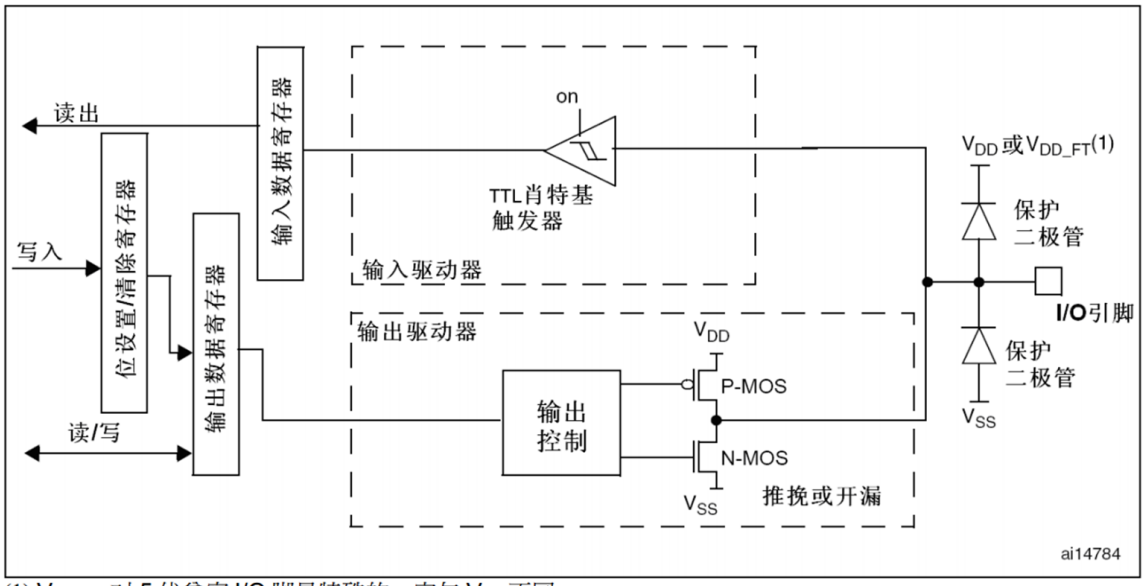
图18　高阻抗的模拟输入配置



(1) V_{DD_FT} 对5伏兼容I/O脚是特殊的，它与V_{DD}不同

7、开漏/推挽输出

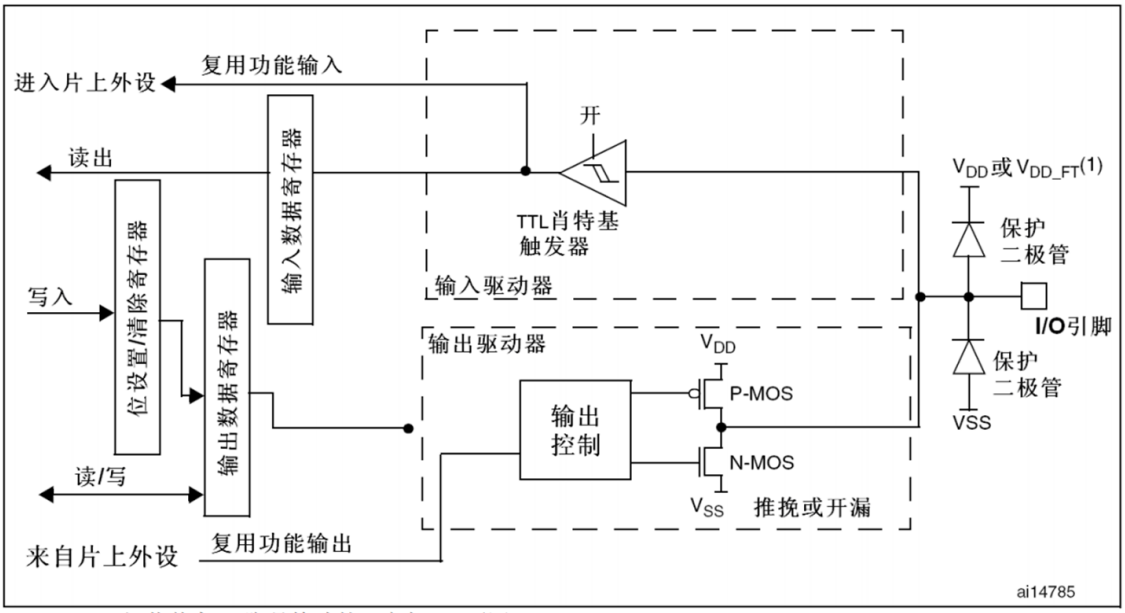
图16　输出配置



(1) V_{DD_FT} 对 5 伏兼容 I/O 脚是特殊的，它与 V_{DD} 不同

8、复用开漏/推挽输出

图17　复用功能配置



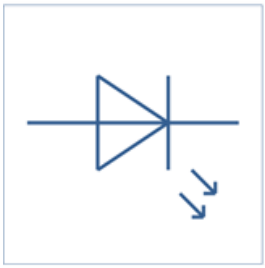
(1) V_{DD_FT} 对5伏兼容I/O脚是特殊的，它与V_{DD}不同

特别说明：在输出的时候是可以有输入（浮空）的，但是在输入模式下，没有输出。

二、LED、蜂鸣器

1、LED：

发光二极管，正向通电点亮，反向通电不亮。

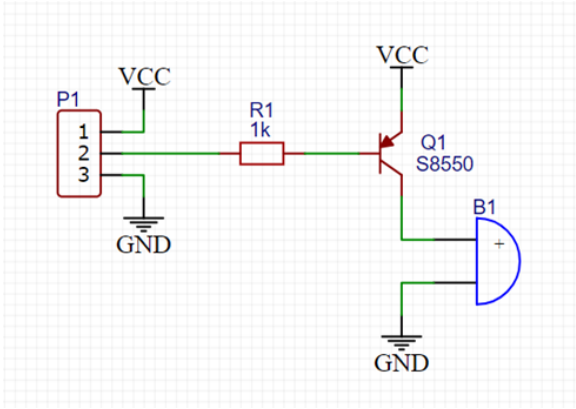




2、蜂鸣器

（1）有源蜂鸣器：内部自带振荡源，将正负极接上直流电压即可持续发声，频率固定。

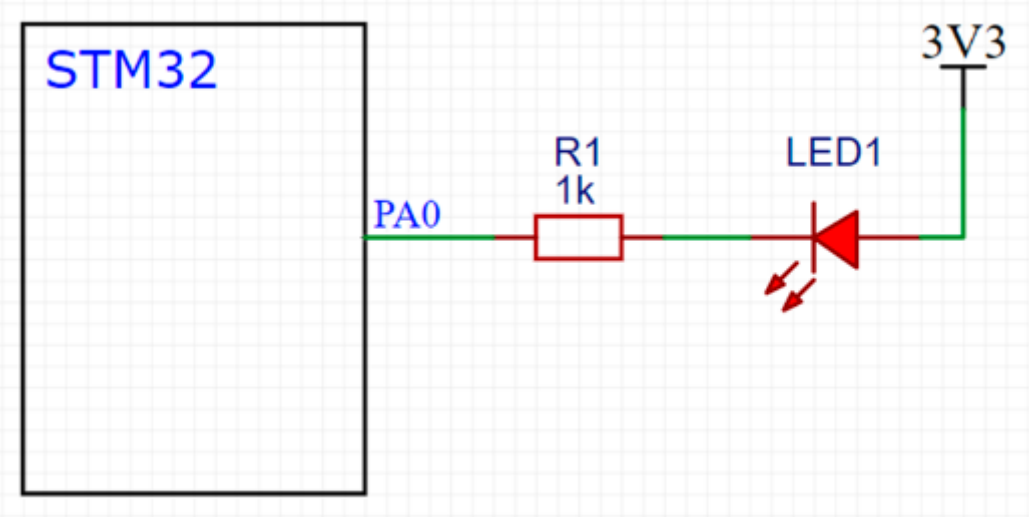
（2）无源蜂鸣器：内部不带振荡源，需要控制器提供振荡脉冲才可发声，调整提供振荡脉冲的频率，可发出不同频率的声。

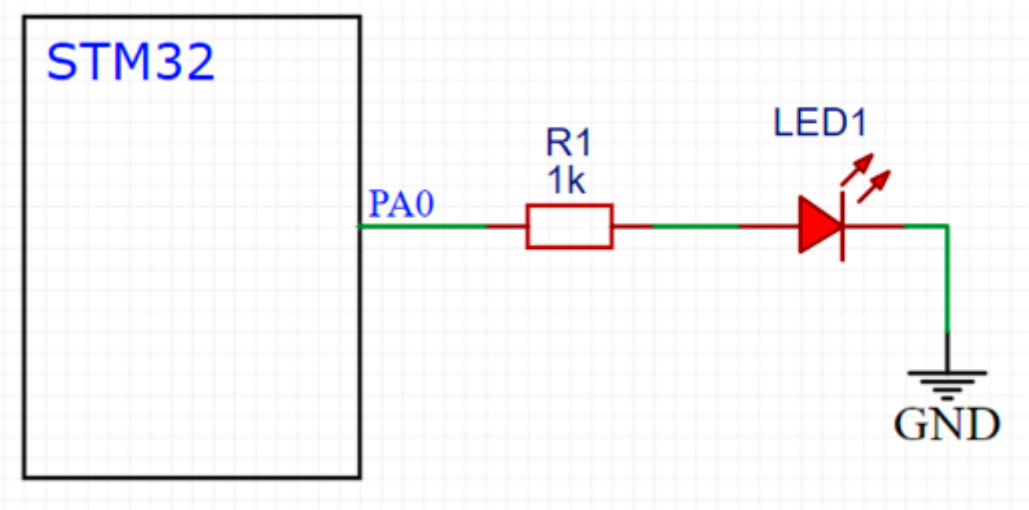




三、基本电路

STM32驱动LED和蜂鸣器的基本电路原理图

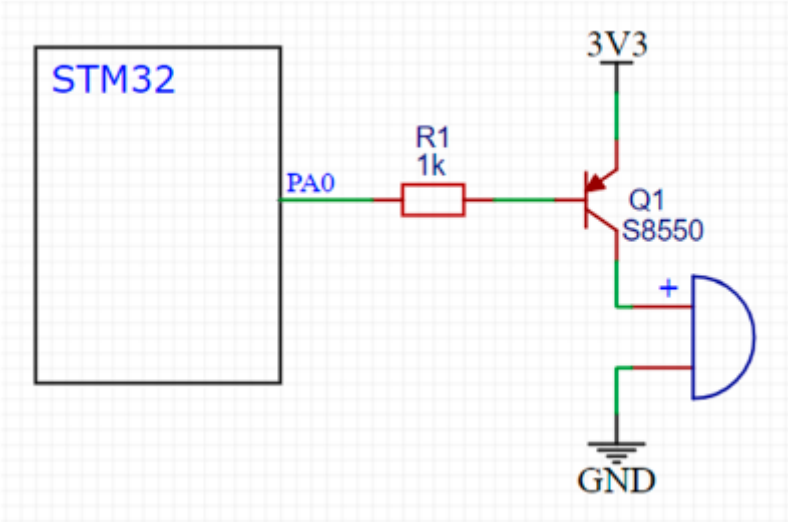


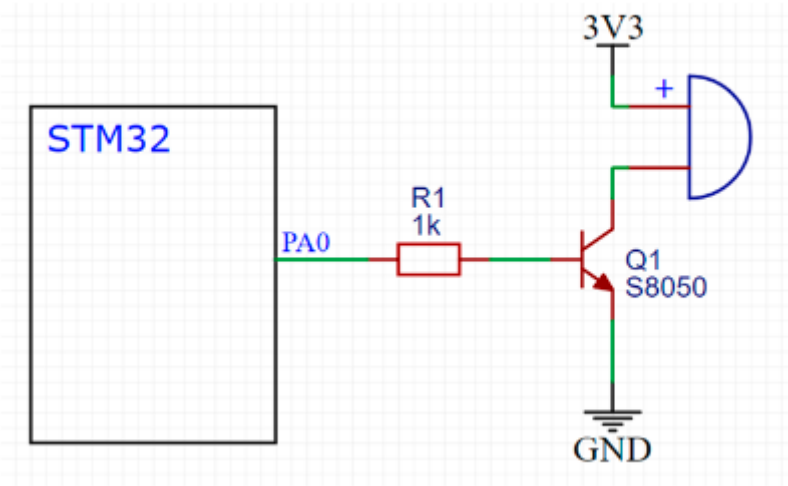


这两个图是使用STM32的GPIO口驱动LED的电路。

其中上面这个是低电平驱动的电路，LED正极接3.3伏，负极通过一个限流电阻接到PA0上。当PA0输出低电平时，LED两端就会产生电压差，就会形成正向导通电流，这样LED就会点亮了。当PA0输出高电平时，因为LED两端都是3.3伏的电压，不会形成电流，所以高电平LED就会熄灭。这里的限流电阻一般都是要接的，一方面它可以防止LED因为电流过大而烧毁，另一方面它也可以调节LED的亮度，如果你觉得LED太亮比较刺眼的话，可以适当的增大限流电阻的阻值。当然，我们实验的时候使用面包板插接LED的话，为了简化电路是可以省去这个限流电阻。大家如果自己设计电路的时候，一定要注意加上。
下面这个图就是高电平驱动的电路，LED负极接到GND，正极通过一个限流电阻接到PA0上，这时就是高电平点亮，低电平熄灭了。那么这两种驱动方式应该如何选择呢？通常就需要考察这个IO口高低电平的驱动能力如何了，前面我们讲过GPIO在推挽输出模式下，高低电平均有比较强的驱动能力，这种情况下这两种接法均可。但是在单片机的电路里，一般倾向使用第一种接法，因为很多单片机或者芯片都使用了**高电平弱驱动**，**低电平强驱动**的规则，这样可以一定程度上避免高低电平打架。所以如果高电平驱动能力弱，那就不能使用第二种连接方法了。
我们再看下面的蜂鸣器电路，这里使用了三极管开关的驱动方案。

> 注意：三极管的通断是需要在发射极和基极之间产生一定的开启电压
>
> - PNP的三极管： 设备最好接在下边
> - NPN的三极管：设备最好接在上边





三极管开关是最简单的驱动电路了，对于功率稍微大一点的直接用IO口驱动会导致STM32负担过重，这时就可以用一个三极管驱动电路来完成驱动的任务。上面这个图是PNP三极管的驱动电路。三极管的左边是基极，带箭头的是发射极，剩下的是集电极。它左边的基极给低电平，三极管就会导通。通过3.3伏和GND就可以给蜂鸣器提供驱动电流了。基极给高电平三极管截止，蜂鸣器就没有电流不会工作了。
下面这个图是NPN三极管的驱动电路。同样左边是基极，带箭头的是发射极，剩下的是集电极。它的驱动逻辑跟上面的是相反的，基极给高电平导通，低电平断开。

另外注意一下，PNP的三极管最好接在上边，NPN的三极管最好接在下边。这是因为三极管的通断是需要在**发射极和基极之间产生一定的开启电压**的，如果你把负载接在发射极这边，可能会导致三极管不能开启了。

四、GPIO输出实验

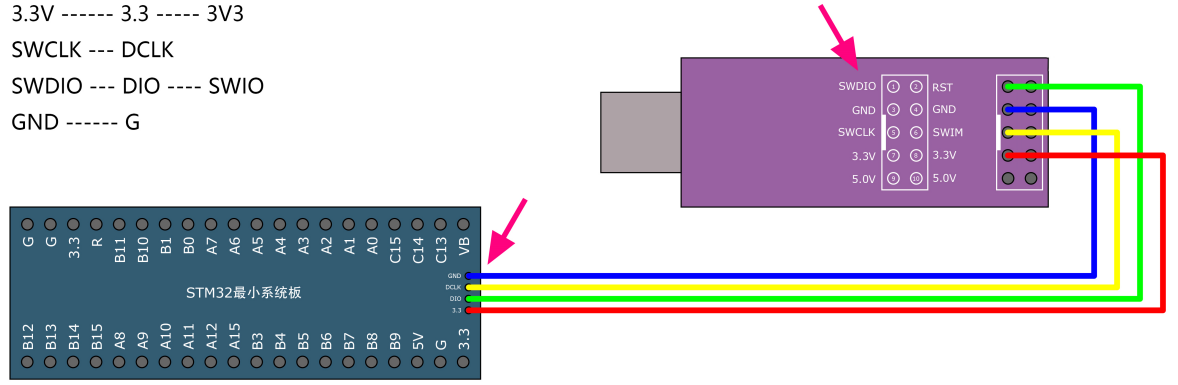
（1）实验目的

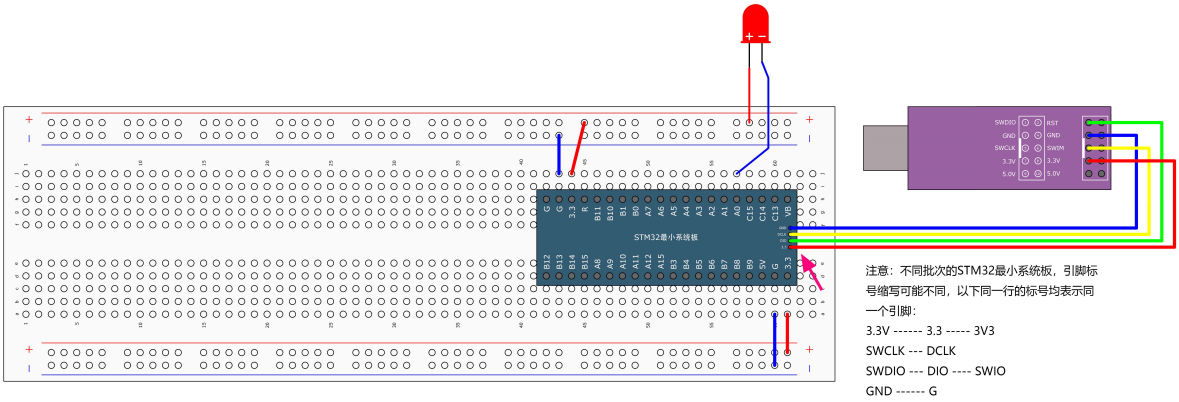
通过对STM32的GPIO口控制实现让LED等闪烁的目的。

（2）硬件电路接线图

注意：不同批次的STM32最小系统板，引脚标号缩写可能不同，以下同一行的标号均表示同一个引脚：
3.3V ------ 3.3 ----- 3V3
SWCLK --- DCLK
SWDIO --- DIO ---- SWIO
GND ------ G

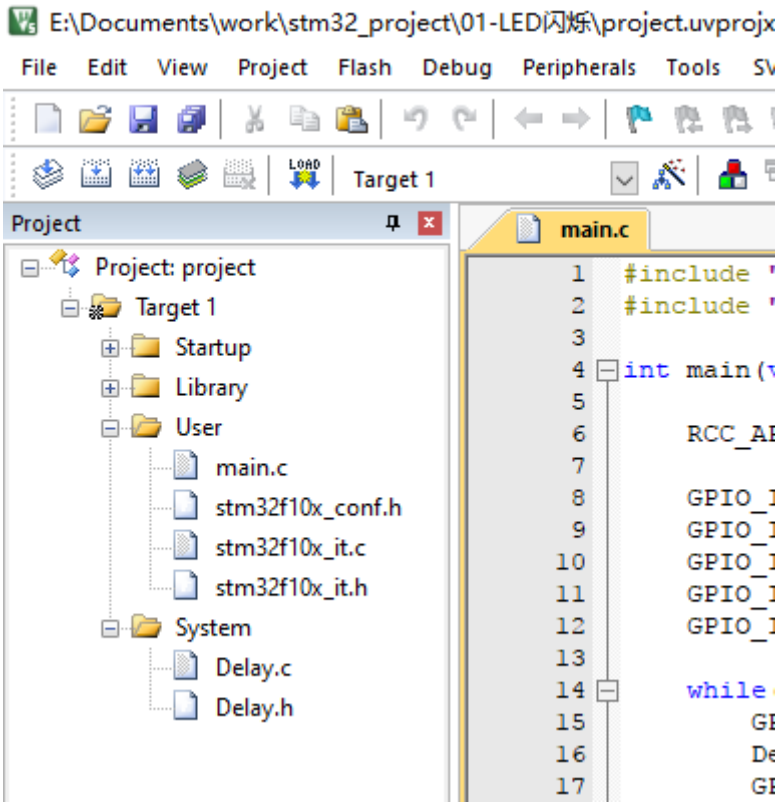
注意：不同批次的STLINK，引脚排序可能不同，请严格按照STLINK壳上指示的引脚标号对应接线，不能只根据引脚位置来进行判断





（3）程序设计

工程结构如下图：



其中System组是与上节课所添加工程不一样的地方，这个组主要提供延时函数，本次课程仅做使用，暂时不展开讲解。除了这个组之外，本实验的工程仅仅是main.c的代码与上节课的工程不一样，具体代码展示如下：

Delay.c

```c
#include "stm32f10x.h"

/**
  * @brief  微秒级延时
  * @param  xus 延时时长，范围：0~233015
  * @retval 无
  */
void Delay_us(uint32_t xus)
{
    SysTick->LOAD = 72 * xus;                //设置定时器重装值
    SysTick->VAL = 0x00;                     //清空当前计数值
    SysTick->CTRL = 0x00000005;              //设置时钟源为HCLK，启动定时器
    while(!(SysTick->CTRL & 0x00010000));    //等待计数到0
    SysTick->CTRL = 0x00000004;              //关闭定时器
}

/**
  * @brief  毫秒级延时
  * @param  xms 延时时长，范围：0~4294967295
  * @retval 无
  */
void Delay_ms(uint32_t xms)
{
    while(xms--)
    {
        Delay_us(1000);
    }
}

/**
  * @brief  秒级延时
  * @param  xs 延时时长，范围：0~4294967295
  * @retval 无
  */
void Delay_s(uint32_t xs)
{
    while(xs--)
    {
        Delay_ms(1000);
    }
}
```

Delay.h

```
#ifndef __DELAY_H
#define __DELAY_H

void Delay_us(uint32_t us);
void Delay_ms(uint32_t ms);
void Delay_s(uint32_t s);

#endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"

int main(void){

    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); // 设置RCC时钟

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; // 设置为推挽输出模式
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_0; // 设置初始化A0引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure); // GPIO初始化

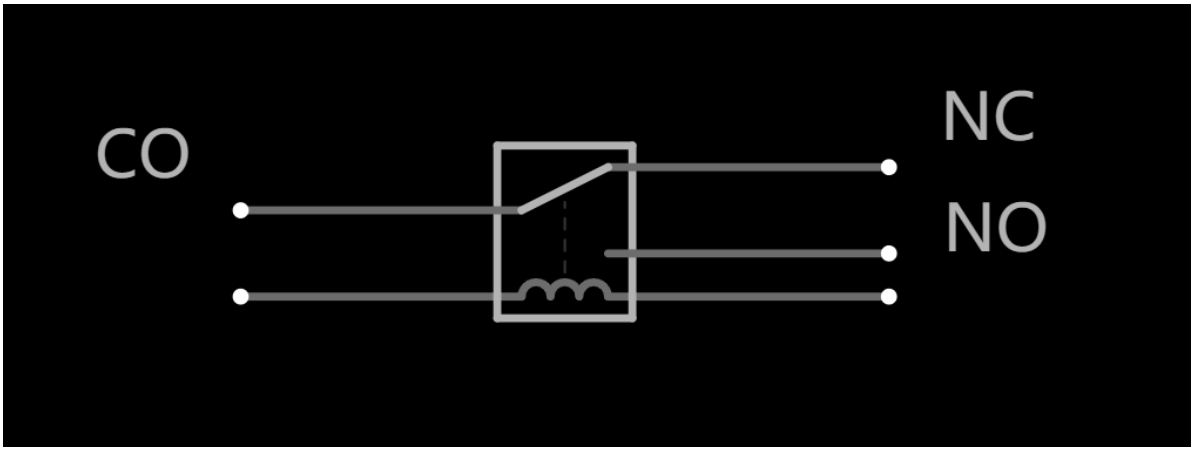
    while(1){
        GPIO_WriteBit(GPIOA, GPIO_Pin_0, Bit_RESET); // A0引脚输出低电平，LED点亮
        Delay_ms(200);
        GPIO_WriteBit(GPIOA, GPIO_Pin_0, Bit_SET); // A0引脚输出高电平，LED熄灭
        Delay_ms(200);
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果是LED等每隔200毫秒间隔闪烁，符合预期设计。

小实验：控制3至5个LED流水灯

五、STM32控制继电器

> NO 常开脚
>
> CO 公共脚
>
> NC 常闭脚



小实验：控制门锁的关闭和断开

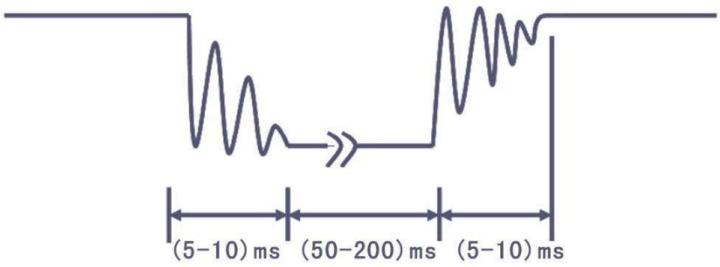
六、按键、t传感器模块

1、按键

按键是常见的输入设备，按下电路导通，松手电路断开。

按键抖动：由于按键内部使用的是机械式弹簧片来进行通断的，所以在按下和松手的瞬间会伴随有一连串的抖动。

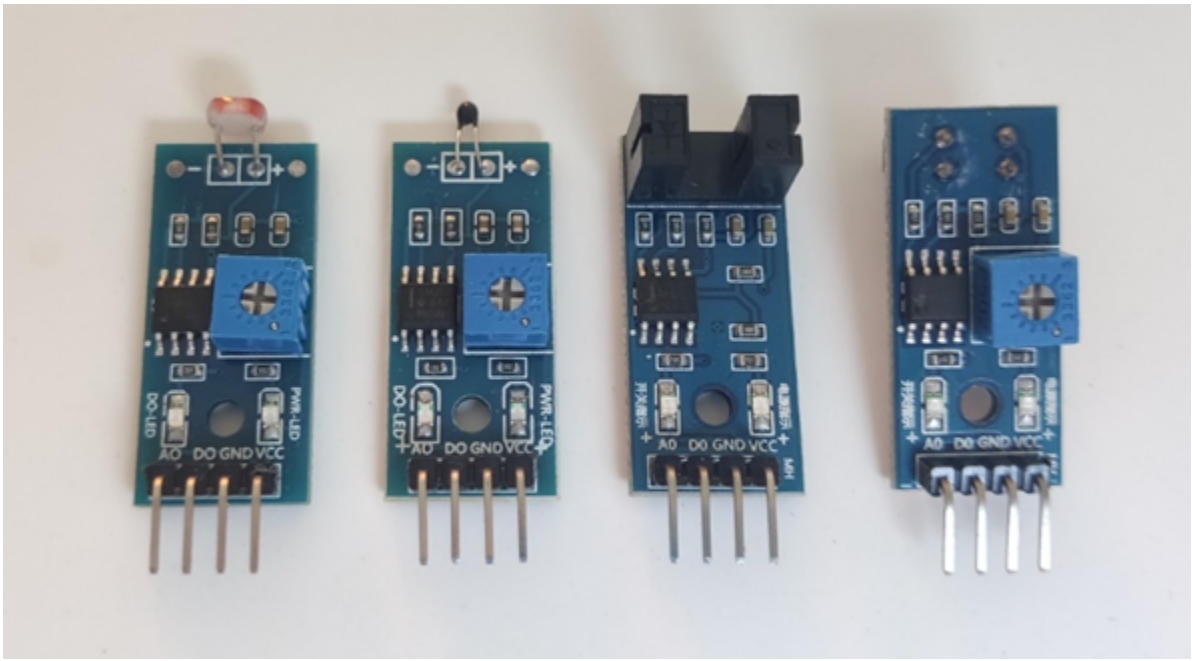


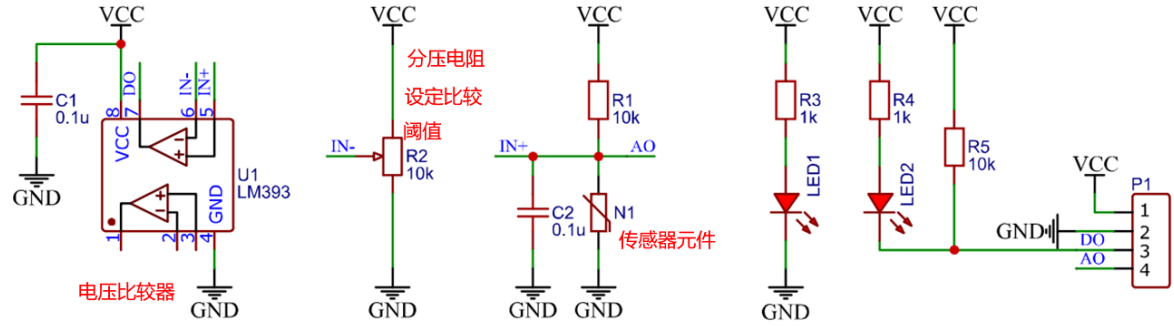


通过上图这个波形就可以看出，假设按键没按下时是高电平，按下就是低电平。那在按键按下的瞬间，信号由高电平变为低电平时就会来回抖动几下，这个抖动会比较快，通常在5-10毫秒之间，人眼是分辨不出来的。但是对于高速运行的单片机而言，5-10毫秒的时间还是很漫长的，所以我们要对这个抖动进行过滤，否则就会出现按键按一下，单片机却反映了按了很多次的现象。另外在按键松手的时候，也会有一小段时间的抖动，这个需要我们在程序中也要注意过滤一下，最简单的过滤方法就是加一段延时，把这个抖动时间耗过去，这样就没有问题了。

2、传感器模块

传感器元件（光敏电阻/热敏电阻/红外接收管等）的电阻会随外界模拟量的变化而变化，通过与定值电阻分压即可得到模拟电压输出，再通过电压比较器进行二值化即可得到数字电压输出。





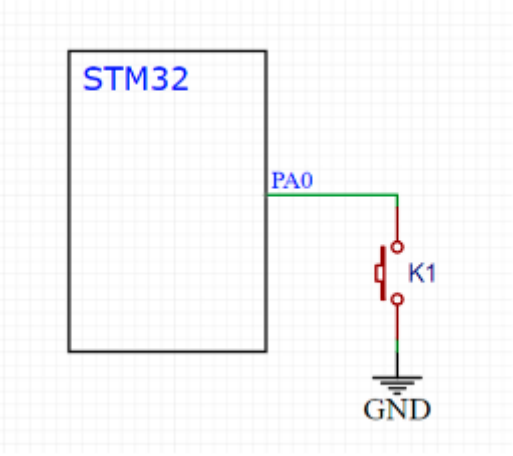
上述四种传感器模块，依次分别是光敏电阻传感器、热敏电阻传感器、对射式红外传感器以及反射式红外传感器。它们的电路结构和工作原理都差不多，这些传感性模块都是利用传感器元件，比如光敏电阻、热敏电阻、红外接收管等，这些元件的电阻会随外界的模拟量变化而变化。比如：光线越强，光敏电阻的阻值就越小；温度越高，热敏电阻的阻值就越小；红外光线越强，红外接收管的阻值就越小。

但是电阻的变化不容易直接被观察，所以我们通常将传感器元件与定值电阻进行串联分压，这样就可以得到模拟电压的输出了，而对电路来说检测电压变化是非常容易的。另外，这个模块还可以通过电压比较器来对这个模拟电压进行二极化，这样就可以得到数字电压输出了。

七、基本电路

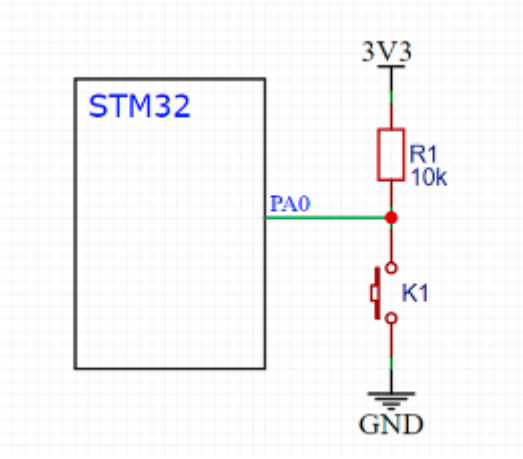
按键和传感器模块的硬件电路。首先按键这里给出了4种接法，前两个是下接按键的方式，后两个是上接按键的方式。一般来说，按键都是用前两种方式，也就是下接的方式。这个原因跟LED的接法类似，是电路设计的习惯和规范。

（1）我们先来看一下第一张图，这种接法是按键的最常用的接法。



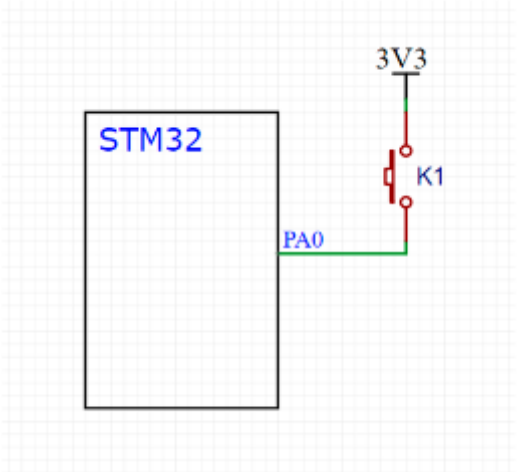
在这里随便选取一个GPIO口，比如PA0，然后通过K1接到地。当按键按下时，PA0被直接接到GND，此时读取PA0口的电压就是低电平；当按键松手时，PA0被悬空，悬空就会导致引脚的电压不确定，所以在这种接法下，必须要求PA0是上拉输入的模式。否则，就会出现引脚电压不确定的错误现象。如果PA0是上拉输入的模式，那么引脚再悬空，PA0就是高电平，所以这种方式下按下按键引脚为低电平，松手引脚为高电平。

（2）接着再看第二张图，相比较上一张图，



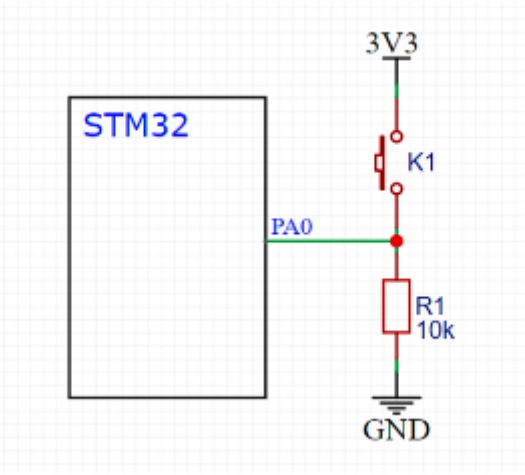
在这里外部接了一个上拉电阻，当按键松手时，引脚由于上拉作用自然保持为高电平，当按键按下时，引脚直接接到GND，所以引脚就为低电平。这种状态下，引脚不会出现悬空状态，所以此时PA0引脚可以配置为浮空输入或者上拉输入。如果是上拉输入，那就是内外两个上拉电阻共同作用，这时高电平就会更强一些，对应高电平就更加稳定。当然这样接的话，当引脚被强行拉到低时，损耗也就会大一些。

（3）第三张图PA0通过按键接到3.3伏，要求PA0必须要配置成下拉输入的模式，当按键按下时引脚为高电平，松手时引脚回到默认值低电平。



这里要求单片机的引脚可以配置为下拉输入的模式，但是一般单片机可能不一定有下拉输入的模式，所以最好还是用上面的接法。

（4）第四张图是最后一种接法，是在第三张图的接法下面再外接一个下拉电阻，

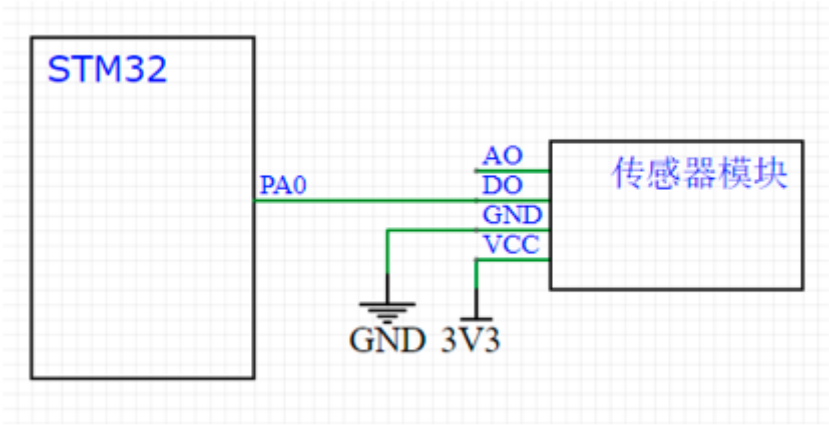


此时PA0需要配置为下拉输入模式或浮空输入模式。

总结一下就是前面的两种接法，按键按下时引脚是低电平，松手是高电平；后面的两种接法，按键按下时是高电平，松手是低电平。1、3两种接法，要求引脚必须配置为上拉或者下拉输入的模式；而2、4两种接法，可以允许引脚是浮空输入的模式，因为已经外置了上拉电阻

和下拉电阻。一般我们都用前面两种接法，而后面两种接法用的比较少。

（5）最后是传感器模块的电路，因为是使用模块的方案，所以电路还是非常简单的。



这里VCC接3.3伏，GND直接接地用于供电，DO数字输出随便接一个端口，比如PA0，用于读取数字量。AO模拟输出在后续课程学习ADC模数转换器的时候再仔细研究，这里就直接空置就好了。

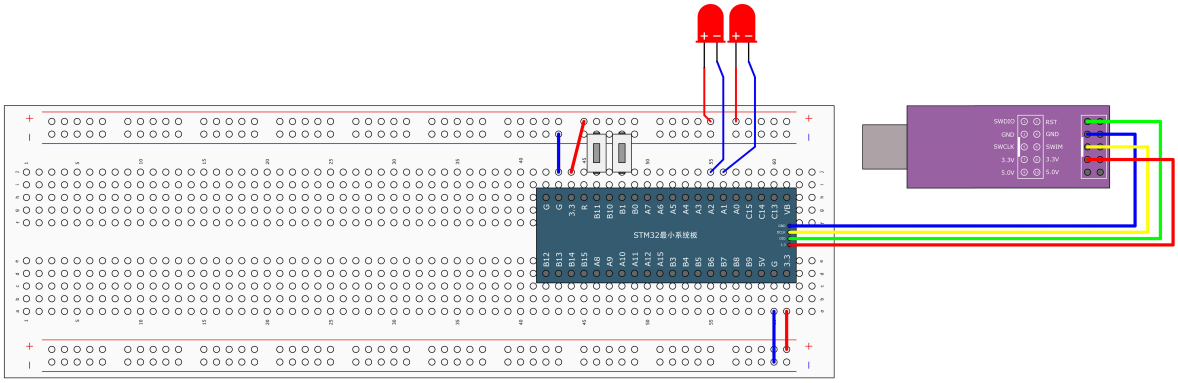
八、GPIO输入实验

1、按键控制LED灯

（1）实验目的

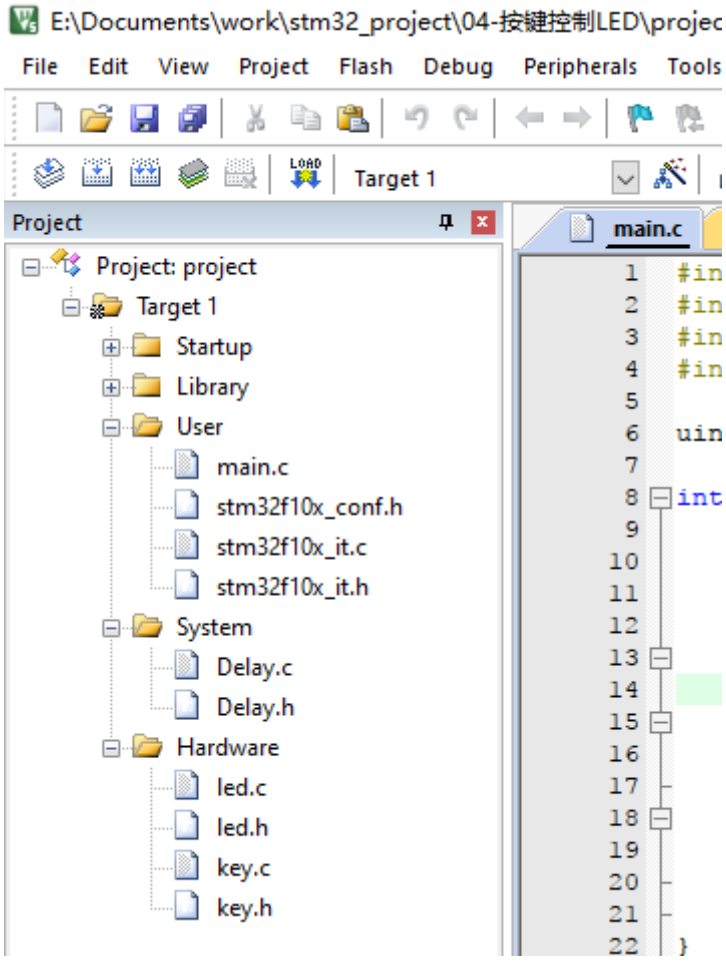
通过对STM32的GPIO口控制实现让按键按下LED点亮，再按一下LED灯熄灭这样的逻辑。具体接几组按键和LED同学们可以自己设计，本案例总共接了2组按键和LED。

（2）硬件电路接线图



（3）程序设计

工程结构如下图



相较于前面的工程这个工程增加了Hardware组，主要目的是将led和按键控制的驱动单独实现，然后在main.c中调用使用，这样可以简化main.c的业务。增加整个工程项目的可读性与可维护性。

在Hardware组中，我们为led以及按键单独定义配置了其GPIO输出和输入的驱动代码，具体代码详情如下：

led.c

```
#include "stm32f10x.h"                  // Device header

void LED_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE); // GPIOA时钟初始化

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP; // 配置为推挽输出模式
```

```
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1 | GPIO_Pin_2; // 同时初始化A0和A1引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOA, &GPIO_InitStructure); // 初始化GPIOA

    GPIO_SetBits(GPIOA, GPIO_Pin_1 | GPIO_Pin_2); // 默认置高，也就是LED初始不亮状态
}

void LED1_ON(void) {
    GPIO_ResetBits(GPIOA, GPIO_Pin_1); // 输出低电平，点亮LED
}

void LED1_OFF(void) {
    GPIO_SetBits(GPIOA, GPIO_Pin_1); // 输出高电平，LED熄灭
}

void LED1_Turn(void) {
    // pin1输出低电平，灯亮
    if (GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_1) == 0) {
        GPIO_SetBits(GPIOA, GPIO_Pin_1); // pin1置高电平，灯灭
    } else {
        GPIO_ResetBits(GPIOA, GPIO_Pin_1); // pin1置低电平，灯亮
    }
}

void LED2_ON(void) {
    GPIO_ResetBits(GPIOA, GPIO_Pin_2);
}

void LED2_OFF(void) {
    GPIO_SetBits(GPIOA, GPIO_Pin_2);
}

void LED2_Turn(void) {
    if (GPIO_ReadOutputDataBit(GPIOA, GPIO_Pin_2) == 0) {
        GPIO_SetBits(GPIOA, GPIO_Pin_2);
    } else {
        GPIO_ResetBits(GPIOA, GPIO_Pin_2);
    }
}

// 允许依据设置指定引脚设置对应的LED亮灭状态
void LED_Set(uint16_t GPIO_Pin, BitAction BitVal) {
    GPIO_WriteBit(GPIOA, GPIO_Pin, BitVal);
}
```

led.h

```
# ifndef __LED_H
# define __LED_H
```

```
void LED_Init(void);
void LED1_ON(void);
void LED1_OFF(void);
void LED1_Turn(void);
void LED2_ON(void);
void LED2_OFF(void);
void LED2_Turn(void);
void LED_Set(uint16_t GPIO_Pin, BitAction BitVal);

# endif
```

key.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"

void KEY_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE); // GPIOB时钟初始化

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU; // 设置上拉输入模式
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_1 | GPIO_Pin_11; // 配置B1和B11引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure); // 初始化GPIOB
}

uint8_t KEY_GetNum(void) {
    uint8_t KeyNum = 0;
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0) {
        Delay_ms(20); // 按键后防抖
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_1) == 0); // 等待按键抬起
        Delay_ms(20); // 按键抬起后防抖
        KeyNum = 1;
    }
    if (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11) == 0) {
        Delay_ms(20);
        while (GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_11) == 0);
        Delay_ms(20);
        KeyNum = 2;
    }
    return KeyNum;
}
```

key.h

```
# ifndef __KEY_H
# define __KEY_H

void KEY_Init(void);
uint8_t KEY_GetNum(void);

# endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "led.h"
#include "key.h"

uint8_t KeyNum;

int main(void){

    LED_Init();
    KEY_Init();

    while(1){
        // 获取按键键值
        KeyNum = KEY_GetNum();
        if (KeyNum == 1) {
            LED1_Turn();
        }
        if (KeyNum == 2) {
            LED2_Turn();
        }
    }
}
```

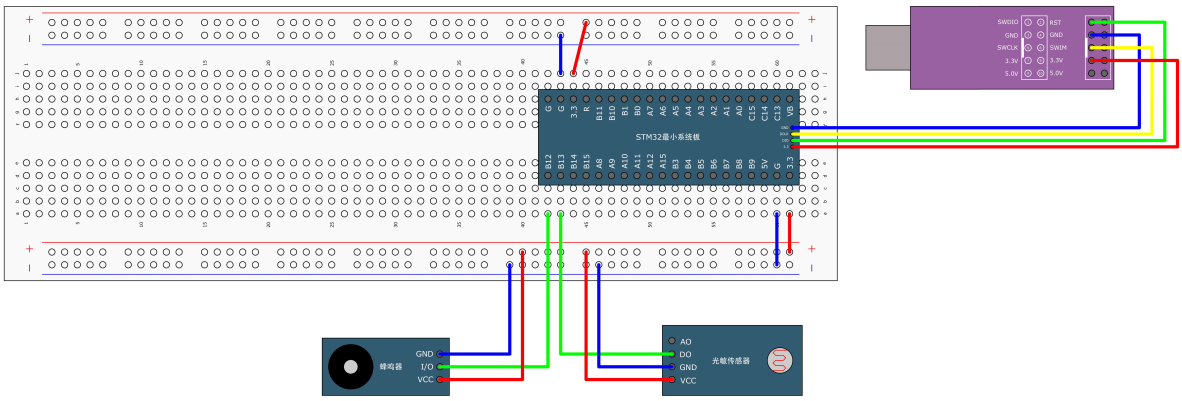
（4）代码完成后，编译并下载到硬件中执行，会发现实验结果能够依据按键控制LED灯的亮灭状态，符合预期设计。

2、光敏传感器控制蜂鸣器

（1）实验目的

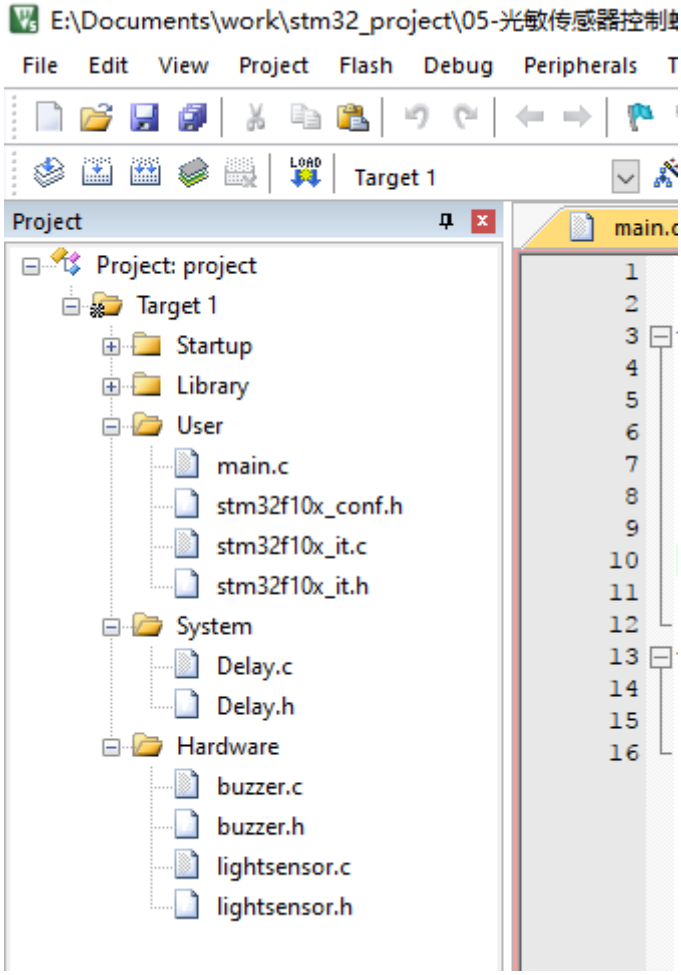
通过对STM32的GPIO口控制实现依据光线强弱来控制蜂鸣器是否发声。当我们用手遮挡光敏传感器的光线时，蜂鸣器发声，而把手拿开，蜂鸣器就不响了。

（2）硬件电路接线图



（3）程序设计

工程结构如下图



相较于前面的工程这个工程只是Hardware组中的外设驱动不一样而已，其他都与前面的工程没有太大不同。下面我们看一下具体的代码设计：

buzzer.c

```
#include "stm32f10x.h"                  // Device header

void Buzzer_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);   // GPIOB时钟初始化

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_Out_PP;     // 设置上拉输入模式
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_12;  // 配置B12引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
```

```
    GPIO_Init(GPIOB, &GPIO_InitStructure);  // 初始化GPIOB

    GPIO_SetBits(GPIOB, GPIO_Pin_12);   // 默认置高，也就是蜂鸣器初始不响状态
}

void Buzzer_ON(void) {
    GPIO_ResetBits(GPIOB, GPIO_Pin_12); // 输出低电平，打开蜂鸣器
}

void Buzzer_OFF(void) {
    GPIO_SetBits(GPIOB, GPIO_Pin_12);   // 输出高电平，关闭蜂鸣器
}

void Buzzer_Turn(void) {
    if (GPIO_ReadOutputDataBit(GPIOB, GPIO_Pin_12) == 0) {
        GPIO_SetBits(GPIOB, GPIO_Pin_12);   // 置高电平，关闭蜂鸣器
    } else {
        GPIO_ResetBits(GPIOB, GPIO_Pin_12); // 置低电平，打开蜂鸣器
    }
}
```

buzzer.h

```
# ifndef __BUZZER_H
# define __BUZZER_H

void Buzzer_Init(void);
void Buzzer_ON(void);
void Buzzer_OFF(void);
void Buzzer_Turn(void);

# endif
```

lightsensor.c

```
#include "stm32f10x.h"                  // Device header

void LightSensor_Init(void) {
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB, ENABLE);   // GPIOB时钟初始化

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;   // 设置上拉输入模式
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_13;  // 配置B13引脚
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB, &GPIO_InitStructure);  // 初始化GPIOB
}

uint8_t LightSensor_Get(void) {
```

```
    return GPIO_ReadInputDataBit(GPIOB, GPIO_Pin_13);   // 读取B13引脚输入电平
}
```

lightsensor.h

```
# ifndef __LIGHT_SENSOR_H
# define __LIGHT_SENSOR_H

void LightSensor_Init(void);
uint8_t LightSensor_Get(void);

# endif
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "buzzer.h"
#include "lightsensor.h"

uint8_t KeyNum;

int main(void){

    Buzzer_Init();
    LightSensor_Init();

    while(1){
        if (LightSensor_Get() == 1) {
            // 光敏传感器感应到光线变暗，在GPIO引脚输入高电平，此时打开蜂鸣器
            Buzzer_ON();
        } else {
            Buzzer_OFF();
        }
    }
}
```

（4）代码完成后，编译并下载到硬件中执行，会发现实验结果能够依据光敏传感器感受光线的强弱不同控制蜂鸣器，符合预期设计。

九、DHT11数字湿温度传感器

参考官方给的文档《DHT11_温湿度.pdf》

小实验：读取温度和湿度并在OLED上显示

课程小结

1. STM32的GPIO是最基本的输入输出，其工作原理需要仔细理解。
2. GPIO的8种输入输出方式时其控制的关键因素，在操作GPIO的时候需要针对其不同的特点在适当的场合下加以运用。
3. LED、蜂鸣器、按键以及各种传感器是最为常见的输入输出外设，对于它们的应用我们需要掌握其不同的电路特性，从而加以运用。
4. GPIO输出和输入分别完成了3个小实验，通过这三个小实验对前面学习的GPIO输入输出控制原理进行实际的验证体会，请各位同学在验证成功后需要对照原理再仔细体会其中的过程。

随堂作业

1. 请解释GPIO的8种输入输出模式。
2. 结合 DHT11_温湿度芯片 控制风扇。

> 设置一个默认阈值比如30 ， 高于30就开风扇，小于等于就关风扇
>
> 这个阈值可以通过两个按键实现加和减。